

Cuaderno de ejercicios

Resuelve en un laboratorio aislado. Las soluciones están en un documento separado. Para cada prueba registra observación, hipótesis, resultado esperado, resultado obtenido y conclusión.

Nivel 1: identificar y leer

E01 - Entradas HTTP

En la petición siguiente enumera todas las entradas controlables y el parser probable de cada una:

```
POST /api/avatar?size=80 HTTP/1.1
Host: perfil.lab
Cookie: session=s-123
Content-Type: application/json

{"url":"http://img.lab/a.png","crop":{"x":0,"y":0}}
```

Indica cuáles podrían seleccionar recurso, identidad, destino de red y parámetros numéricos.

E02 - Source y sink

```
name = request.args["name"]
safe = name.replace(";", "")
result = subprocess.run("lookup " + safe, shell=True, capture_output=True)
```

Marca source, transformación, validación, intérprete y sink. Explica qué propiedad intenta garantizar el reemplazo y por qué esa propiedad es insuficiente.

E03 - Leer una respuesta

Una petición normal devuelve 200, 4.120 bytes y 85 ms. Una comilla devuelve 500, 780 bytes y 82 ms. Una segunda comilla también devuelve 500. Clasifica el hallazgo y diseña un control que permita distinguir un error SQL de otro error de aplicación.

E04 - Puerto y servicio

Un escaneo muestra TCP/8080 abierto. Redacta tres hipótesis y una prueba mínima para cada una. No uses el nombre de puerto como confirmación.

Nivel 2: construir pruebas

E05 - SQL por contexto

Compara:

```
SELECT * FROM items WHERE id = INPUT
SELECT * FROM items WHERE name = 'INPUT'
```

Explica por qué la misma comilla no tiene la misma función. Diseña pares verdadero/falso sin extraer datos.

E06 - Command o argument injection

Escenario A ejecuta `sh -c "ping -c 1 " + host`. Escenario B ejecuta `execve("ping", ["ping", "-c", "1", host])`. Predice el significado de `127.0.0.1; id` en ambos. Diseña una prueba inocua para distinguirlos.

E07 - Traversal

La aplicación abre `/srv/reports/` más el valor `file`. El error revela `/srv/reports/2026/julio/no.txt`. Construye la ruta relativa mínima para solicitar `/srv/marker.txt` y muestra la normalización paso a paso.

E08 - SSRF con callback

Hay un parámetro `url`. Diseña una prueba HTTP hacia un listener propio, un control negativo y la evidencia mínima. Indica qué verificarías antes de probar loopback.

E09 - Upload por capas

Diseña una matriz de pruebas que cambie una sola variable: nombre, Content-Type, bytes, extensión almacenada y ruta de servicio. El objetivo es saber qué capa decide la aceptación y cuál la interpretación.

E10 - SSTI

La cadena `{{7*7}}` aparece como 49; `{{7*'7'}}` aparece como siete repeticiones. Explica qué está confirmado y qué no. Indica el siguiente paso basado en identificación del motor, no en una cadena RCE copiada.

Nivel 3: adaptar y descartar

E11 - Normalización

Un filtro rechaza la cadena literal `../`; el backend registra que `%2e%2e%2f` llega como `../`. Dibuja el orden de validación y decodificación. Propón una defensa correcta y una prueba para verificarla.

E12 - Falso positivo temporal

Cinco peticiones de control tardan 100, 140, 900, 120 y 110 ms. Una prueba temporal tarda 820 ms una vez. ¿Qué conclusión es válida? Diseña una medición mejor.

E13 - Autorización

Dos usuarios pueden solicitar `/api/invoices/41`. Usuario A obtiene 200; usuario B obtiene 403. Al cambiar a `/api/invoices/42`, B obtiene 200. Explica qué debes comprobar antes de afirmar IDOR y qué evidencia confirmaría acceso cruzado.

E14 - sudo

`sudo -l` permite `/usr/bin/find` como root sin contraseña. Sin ejecutar una shell, describe cómo verificarías la regla, versión, comportamiento relevante e identidad efectiva. Explica por qué una receta de otro binario no es transferible.

E15 - Servicio Windows

`sc qc DemoSvc` muestra `C:\Program Files\Demo App\service.exe` sin comillas y cuenta LocalSystem. ¿Es suficiente para confirmar escalada? Enumera permisos y condiciones adicionales necesarios.

E16 - Kerberos

Un ccache válido falla con `KRB_AP_ERR_SKEW`. Formula hipótesis, prueba mínima y adaptación. Distingue autenticación fallida por hora de autorización denegada.

Nivel 4: casos completos

E17 - Burp Repeater

Una API acepta JSON y devuelve campos diferentes según rol. Diseña una secuencia de seis peticiones que estudie autenticación, autorización de objeto, método, campo client-side y sesión, cambiando una variable cada vez.

E18 - Terminal Linux

Tienes shell como `www-data`. Construye un plan de enumeración basado en evidencia para `sudo`, `SUID`, `capabilities`, `cron`, grupos y secretos. Para cada categoría indica salida que haría avanzar y salida que la descartaría.

E19 - Room tipo TryHackMe

Observas 22/SSH, 80/HTTP y 445/SMB. La web redirige a un nombre que no resuelve; SMB permite sesión nula pero no lista shares. Prioriza diez pruebas y justifica el orden. Incluye criterios para abandonar cada hipótesis.

E20 - Encadenamiento y falso positivo

Una LFI lee un archivo de configuración con una contraseña; esa contraseña funciona en SSH para un usuario sin `sudo`. Un `cron` de root ejecuta un script legible pero no escribible en un directorio escribible. Construye la cadena de hipótesis. Identifica qué es confirmación, qué es solo indicio y qué permisos debes verificar antes de afirmar impacto.

Preguntas tipo test

1. ¿Qué confirma mejor una SQLi booleana? A) un 500; B) un par verdadero/falso reproducible; C) una página lenta; D) una versión en cabecera.
2. ¿Qué capa suele consumir #fragment de una URL HTTP? A) servidor; B) DNS; C) cliente; D) TLS.
3. En `execve` con `argv` fijo, `;` es normalmente: A) separador shell; B) dato dentro de un argumento; C) pipe; D) redirección.
4. Una arista BloodHound es: A) impacto confirmado; B) ruta potencial que requiere precondiciones; C) contraseña; D) ticket.
5. La mejor defensa ante traversal es: A) quitar `../`; B) doble URL decode; C) IDs indirectos y ruta canónica confinada; D) bloquear puntos.